Zaawansowane zagadnienia algorytmiki i programowania

dr hab. inż. Agnieszka Duraj

Projekty laboratoryjne – opis dla studentów

Celem projektów jest praktyczna implementacja wybranych algorytmów, ich poprawna weryfikacja oraz eksperymentalna analiza wydajności. Każdy student lub zespół realizuje jeden wybrany projekt zgodnie z wymaganiami przedstawionymi w dokumencie.

Wymagania wspólne dla wszystkich projektów

- Projekt składa się z czterech etapów dokumentowanych w czterech raportach.
- Kluczowe algorytmy wskazane w projekcie należy zaimplementować samodzielnie. Gotowe implementacje biblioteczne mogą być wykorzystane jedynie pomocniczo, np. do weryfikacji wyników, jeśli prowadzący na to zezwoli.
- Kod źródłowy należy przekazać oddzielnie. Raport powinien zawierać opis rozwiązania, pseudokod lub schemat działania oraz najważniejsze fragmenty implementacji, ale nie wielostronicowe listingi kodu.
- Każda implementacja musi zostać zweryfikowana testami poprawności, w tym przypadkami typowymi i brzegowymi.
- Eksperymenty należy wykonywać dla kilku rozmiarów i/lub typów danych. Pomiary czasowe powinny być powtarzane, a sposób agregacji wyników (np. średnia lub mediana) opisany w raporcie.
- Należy podać środowisko eksperymentalne: język programowania, wersje istotnych bibliotek, system operacyjny oraz podstawowe informacje o sprzęcie.
- Raporty muszą zawierać tabele i czytelne wykresy. Każdy wykres powinien mieć podpisane osie, jednostki i legendę, jeśli jest potrzebna.
- W raporcie końcowym należy porównać wyniki eksperymentalne z przewidywaniami wynikającymi ze złożoności teoretycznej i omówić ograniczenia przeprowadzonych badań.

Raport	Główny zakres
Raport 1	Podstawy teoretyczne, pytania badawcze i plan eksperymentów
Raport 2	Implementacja podstawowa i testy poprawności
Raport 3	Implementacja rozszerzona i eksperymenty
Raport 4	Analiza porównawcza, wizualizacja wyników i wnioski

Zaawansowane zagadnienia algorytmiki i programowania

dr hab. inż. Agnieszka Duraj

Projekt 1. Algorytmy grafowe – analiza algorytmów najkrótszej ścieżki

Cel projektu: Implementacja i eksperymentalne porównanie algorytmów wyznaczania najkrótszych ścieżek w grafach o różnych właściwościach. Należy zbadać wpływ rozmiaru grafu, jego gęstości, rodzaju wag oraz sposobu reprezentacji na czas działania i zapotrzebowanie pamięciowe algorytmów.

Algorytmy: Dijkstra, Bellman-Ford, A* oraz Floyd-Warshall.

Raport 1. Podstawy teoretyczne i plan eksperymentów

Raport powinien zawierać:

- definicje grafu, ścieżki, kosztu ścieżki i najkrótszej ścieżki;
- rozróżnienie problemów single-source shortest path i all-pairs shortest paths;
- opis zasady działania algorytmów Dijkstry, Bellmana-Forda, A* i Floyd-Warshalla;
- pseudokod lub schemat działania każdego algorytmu;
- złożoność czasową i pamięciową oraz najważniejsze ograniczenia metod;
- porównanie listy sąsiedztwa i macierzy sąsiedztwa;
- opis sposobu generowania lub pozyskania grafów testowych;
- plan eksperymentów obejmujący co najmniej kilka rozmiarów grafu oraz grafy o różnej gęstości;
- opis mierzonych wielkości: czas wykonania, pamięć oraz inne uzasadnione charakterystyki;
- co najmniej 2 pytania badawcze, na które student odpowie w raporcie końcowym.

Raport 2. Implementacja podstawowa – Dijkstra i Bellman-Ford

- opis własnej implementacji obu algorytmów i zastosowanych struktur danych;
- uzasadnienie najważniejszych decyzji implementacyjnych;
- testy poprawności na małych grafach z wynikiem możliwym do ręcznej weryfikacji;
- testy przypadków szczególnych, np. brak ścieżki, graf niespójny i ujemne wagi tam, gdzie są dopuszczalne;
- porównanie wyników algorytmów dla wspólnych przypadków testowych;
- wstępne pomiary czasu działania;
- krótką analizę zaobserwowanych różnic.

Zaawansowane zagadnienia algorytmiki i programowania

dr hab. inż. Agnieszka Duraj

Raport 3. A* i Floyd-Warshall – eksperymenty

- opis implementacji A* i Floyd-Warshalla;
- dla A*: definicję zastosowanej heurystyki, jej uzasadnienie oraz warunki, w których może być stosowana;
- weryfikację poprawności wszystkich czterech algorytmów;
- eksperymenty dla kilku rozmiarów grafów oraz grafów rzadkich i gęstych;
- pomiary czasu działania;
- pomiar pamięci lub uzasadnioną analizę zapotrzebowania pamięciowego;
- zbiorczą tabelę wyników;
- co najmniej dwa wykresy, np. czas vs liczba wierzchołków oraz czas vs gęstość grafu.

Raport 4. Analiza porównawcza i wnioski

- końcowe zestawienie wyników wszystkich algorytmów;
- porównanie wyników eksperymentalnych ze złożonością teoretyczną;
- analizę wpływu rozmiaru i gęstości grafu;
- analizę wpływu reprezentacji grafu;
- wskazanie warunków, w których poszczególne algorytmy są właściwym wyborem;
- wizualizację przykładowego grafu i znalezionej najkrótszej ścieżki;
- odpowiedzi na pytania badawcze z Raportu 1;
- ograniczenia eksperymentu oraz wnioski końcowe.

Zaawansowane zagadnienia algorytmiki i programowania

dr hab. inż. Agnieszka Duraj

Projekt 2. Zaawansowane struktury danych – analiza efektywności operacji

Cel projektu: Implementacja i eksperymentalna analiza zaawansowanych struktur danych dla operacji insert, search i delete. Należy zbadać, w jaki sposób liczba elementów, kolejność danych oraz charakter wykonywanych operacji wpływają na rzeczywistą wydajność struktur.

Struktury obowiązkowe: AVL Tree i Splay Tree. Trzecia struktura do wyboru: B-tree albo tablica hashująca.

Raport 1. Podstawy teoretyczne i projekt eksperymentu

- krótkie przypomnienie właściwości drzewa BST;
- zasadę równoważenia AVL i opis rotacji;
- zasadę działania Splay Tree i operacji splay;
- opis wybranej trzeciej struktury: B-tree albo tablicy hashującej;
- złożoność operacji insert, search i delete;
- wyjaśnienie pojęcia złożoności amortyzowanej tam, gdzie ma zastosowanie;
- analizę możliwego wpływu uporządkowania danych wejściowych;
- plan własnej implementacji;
- plan eksperymentów dla co najmniej trzech rodzajów danych wejściowych, np. losowych, uporządkowanych i częściowo uporządkowanych;
- co najmniej 2 pytania badawcze.

Raport 2. Implementacja podstawowa – AVL i Splay Tree

- opis klas i struktur wykorzystanych w implementacji;
- opis implementacji insert, search i delete;
- opis rotacji AVL;
- opis operacji splay;
- testy poprawności każdej operacji;
- testy przypadków brzegowych, np. pusta struktura, element nieistniejący, powtarzające się klucze – zgodnie z przyjętą polityką;
- wizualizację przykładowych struktur po wykonaniu wybranej sekwencji operacji;
- wstępne pomiary czasu.

Zaawansowane zagadnienia algorytmiki i programowania

dr hab. inż. Agnieszka Duraj

Raport 3. Trzecia struktura i eksperymenty

- opis i implementację B-tree albo tablicy hashującej;
- uzasadnienie parametrów wybranej struktury, np. stopnia B-tree lub sposobu rozwiązywania kolizji;
- charakterystykę danych testowych;
- osobne pomiary czasu operacji insert, search i delete;
- eksperymenty dla kilku rozmiarów danych;
- eksperymenty dla różnych kolejności danych wejściowych;
- powtórzenie pomiarów i raportowanie co najmniej średniego lub medianowego czasu;
- pomiar pamięci lub uzasadnione oszacowanie kosztu pamięciowego;
- tabele wyników i wykresy porównawcze.

Raport 4. Analiza i wnioski

- końcową tabelę porównawczą struktur;
- analizę efektywności insert, search i delete;
- analizę wpływu uporządkowania danych;
- porównanie wyników eksperymentalnych ze złożonością teoretyczną;
- analizę kosztu utrzymywania i reorganizacji struktur;
- wskazanie typów zastosowań, dla których poszczególne struktury są odpowiednie;
- odpowiedzi na pytania badawcze;
- ograniczenia eksperymentu oraz wnioski końcowe.

Zaawansowane zagadnienia algorytmiki i programowania

dr hab. inż. Agnieszka Duraj

Projekt 3. Algorytmy teorii liczb i ich zastosowanie w RSA

Cel projektu: Implementacja i analiza efektywności wybranych algorytmów teorii liczb wykorzystywanych w RSA oraz zbadanie wpływu rozmiaru danych na ich koszt obliczeniowy. Implementacja RSA ma charakter edukacyjny i nie może być traktowana jako bezpieczna implementacja produkcyjna.

Raport 1. Podstawy teoretyczne i plan eksperymentów

- podstawy arytmetyki modularnej;
- NWD oraz algorytm Euklidesa;
- rozszerzony algorytm Euklidesa i jego zastosowanie;
- szybkie potęgowanie modularne;
- pojęcie liczby pierwszej i ideę testów pierwszości;
- zasadę działania probabilistycznego testu Millera-Rabina;
- ogólną zasadę działania RSA: generowanie kluczy, szyfrowanie i deszyfrowanie;
- złożoność analizowanych algorytmów;
- plan implementacji i plan eksperymentów dla różnych rozmiarów liczb/kluczy;
- co najmniej 2 pytania badawcze.

Raport 2. Implementacja algorytmów podstawowych

- Student samodzielnie implementuje algorytm Euklidesa, rozszerzony algorytm Euklidesa oraz szybkie potęgowanie modularne.
- pseudokod i opis implementacji każdego algorytmu;
- testy poprawności;
- porównanie prostego i szybkiego sposobu potęgowania modularnego;
- eksperymenty dla różnych rozmiarów danych;
- tabelę wyników;
- wykres czasu działania w zależności od rozmiaru danych;
- krótką interpretację wyników.

Zaawansowane zagadnienia algorytmiki i programowania

dr hab. inż. Agnieszka Duraj

Raport 3. Miller-Rabin i RSA

- Student implementuje test Millera-Rabina, generowanie liczb pierwszych, generowanie pary kluczy RSA oraz operacje szyfrowania i deszyfrowania.
- opis implementacji testu Millera-Rabina;
- wyjaśnienie znaczenia liczby rund/iteracji testu;
- opis procesu generowania kluczy RSA;
- demonstrację poprawności szyfrowania i deszyfrowania;
- pomiary czasu generowania kluczy dla kilku długości;
- pomiary czasu szyfrowania i deszyfrowania;
- tabele i wykresy wyników;
- krótką analizę kosztu obliczeniowego.

Raport 4. Analiza eksperymentalna i wnioski

- zbiorczą tabelę wyników;
- wykresy czasu działania w zależności od rozmiaru danych;
- analizę wpływu długości klucza na czas generowania kluczy;
- analizę wpływu długości klucza na koszt szyfrowania i deszyfrowania;
- porównanie obserwacji eksperymentalnych z teorią;
- omówienie ograniczeń własnej implementacji;
- jasne zaznaczenie, że rozwiązanie jest demonstracją edukacyjną, a nie implementacją przeznaczoną do zastosowań produkcyjnych;
- odpowiedzi na pytania badawcze i wnioski końcowe.

Zaawansowane zagadnienia algorytmiki i programowania

dr hab. inż. Agnieszka Duraj

Projekt 4. Algorytmy wyszukiwania wzorców w tekście

Cel projektu: Implementacja i eksperymentalne porównanie algorytmów wyszukiwania pojedynczych i wielu wzorców oraz analiza wpływu długości tekstu, długości wzorca, liczby wzorców i właściwości alfabetu na ich wydajność.

Algorytmy: wyszukiwanie naiwne jako baseline, KMP, Rabin-Karp, Boyer-Moore oraz Aho-Corasick.

Raport 1. Podstawy teoretyczne i projekt eksperymentu

- definicję problemu wyszukiwania wzorca w tekście;
- opis wyszukiwania naiwnego jako punktu odniesienia;
- zasadę działania KMP, Rabin-Karp, Boyer-Moore i Aho-Corasick;
- pseudokod lub schemat działania metod;
- opis preprocessingu wykonywanego przez poszczególne algorytmy;
- złożoność czasową i pamięciową;
- różnicę między wyszukiwaniem pojedynczego i wielu wzorców;
- plan badań uwzględniający długość tekstu, długość wzorca, liczbę wzorców oraz rodzaj danych;
- co najmniej 2 pytania badawcze.

Raport 2. Implementacja podstawowa – baseline, KMP i Rabin-Karp

- opis własnej implementacji wyszukiwania naiwnego, KMP i Rabin-Karp;
- opis wykorzystanych struktur danych i preprocessingu;
- testy poprawności dla małych danych;
- przypadki: brak wzorca, jedno wystąpienie, wiele wystąpień oraz nakładające się wystąpienia;
- testy przypadków brzegowych, np. pusty tekst lub wzorzec – zgodnie z przyjętą specyfikacją;
- wstępne pomiary czasu;
- porównanie poprawności i wydajności zaimplementowanych metod.

Zaawansowane zagadnienia algorytmiki i programowania

dr hab. inż. Agnieszka Duraj

Raport 3. Boyer-Moore i Aho-Corasick – eksperymenty

- opis i implementację Boyer-Moore oraz Aho-Corasick;
- weryfikację poprawności wszystkich algorytmów;
- eksperymenty dla różnych długości tekstu;
- eksperymenty dla różnych długości wzorca;
- analizę wpływu wielkości alfabetu;
- dla Aho-Corasick: eksperymenty dla różnej liczby jednocześnie wyszukiwanych wzorców;
- wykorzystanie co najmniej dwóch typów danych, np. tekstu naturalnego i sekwencji DNA;
- oddzielny pomiar czasu preprocessingu i wyszukiwania, jeśli ma to znaczenie dla danej metody;
- tabele wyników i co najmniej dwa wykresy.

Raport 4. Analiza porównawcza i wnioski

- końcowe zestawienie wyników wszystkich algorytmów;
- porównanie czasu preprocessingu i czasu właściwego wyszukiwania;
- analizę wyników dla krótkich i długich wzorców;
- analizę wpływu alfabetu i charakteru danych;
- porównanie wyszukiwania pojedynczego i wielu wzorców;
- porównanie wyników eksperymentalnych ze złożonością teoretyczną;
- wskazanie warunków, w których poszczególne algorytmy są szczególnie użyteczne;
- odpowiedzi na pytania badawcze;
- ograniczenia eksperymentu i wnioski końcowe.

Sposób oddawania projektu

- Każdy raport należy przekazać w terminie wskazanym przez prowadzącego.
- Kod źródłowy należy przekazywać w osobnym pliku lub repozytorium zgodnie z instrukcją prowadzącego.
- Kod powinien być uruchamialny, uporządkowany i zawierać instrukcję uruchomienia, jeśli jest ona potrzebna.
- Wyniki przedstawione w raporcie muszą pochodzić z przekazanej implementacji.
- Student powinien być przygotowany do wyjaśnienia sposobu działania kodu, przeprowadzonych eksperymentów oraz interpretacji uzyskanych wyników.